Pandas - Dataframe & Series

→ add pandas to req. txt

```
>>> import pandas as pd
# pandas series like 1d array
>>> data = [1,2,3,4,5]
>>> series = pd.Series(data)
>>> print(series)
```

(from list)

→ series

indices	series
0	1
1	2
2	3
3	4
4	5

dtype: int64

```
>>> data = {'a':1, 'b':2, 'c':3}
ser = pd.Series(data)
print(ser)
```

(from dict)

→

a	1
b	2
c	3

dtype: int64

keys become index

values become series values

Series

④ separate index provided

```
>>> data = [10, 30, 50]
    index = ['a', 'b', 'c']

    ser = pd.Series(data, index=index)

print(series)
```

```
a    10
b    30
c    50
```

Dataframe

→ Creation using a dict

```
>>> data = {'Name': ['A', 'B', 'C'],
            'Age': [1, 2, 3],
            'City': ['x', 'y', 'z']}

>>> df = pd.DataFrame(data)
    print(df)
    print(type(df))
```

→

	Name	Age	City
0	A	1	x
1	B	2	y
2	C	3	z

<class 'pandas.core.frame.DataFrame'>

Note: If needed, you can convert df into array

```
>>> a = np.array(df)
```

(will skip index and column names)

Creating df from a list of dictionaries.

Same way

df = pd.DataFrame(data)

To read a csv file.

df = pd.read_csv(file-name)

Note: To see first 5 records etc.

>>> df.head() → top 5 records

df.tail() → last 5 records

---

df.loc[ ] → usage → df.loc[:, ['Name', 'age']]

selecting columns

selecting rows

↳ uses labels

df.loc['x'] or [1]

In a df, labels are names, positions are indexes.

df.iloc[ ] →

↳ uses positions.

>>> df.at[2, 'Name'] → Giving value in 3rd row in column 'Name'.

⊛ Accessing a specified element ~~at~~ using (iat)

>>> df.iat[2,2] → (exact indexation)

row index — column index

Data Manipulation

>>> df['Salary'] = [50, 60, 70] (Adding a column)

→ adds a column with give values.

>>> df.drop('Salary', axis = 1, inplace = True)

- df → dataframe name/identifier
- drop → delete/drop operation
- 'Salary' → name of column to drop
- axis = 1 → by default, axis = 0 ie, row index
- inplace = True → makes permanent changes to df

⊛ deleting a column

Updating some column

>>> df['Age'] = df['Age'] + 1

- df['Age'] → saves it into the original 'age' column
- df['Age'] + 1 → returns 'Age' updated

Deleting a record

```
df.drop(0, axis=0, inplace=True)
```

→ by default, axis = 0 ie for rows.

```
df.drop(0, inplace=True)
```

→ Drops the 1st record permanently.

```
df.describe()
```

→ gives the statistical summary.

---

loc, at, iloc, iat

loc / iloc → many values
at / iat → single value

```
df.at['x', 'Age']   → 20
```
→ used labels

```
df.iat[0, 1]   → 20
```
→ used indices

```
df.loc[:, ['Name']]
```

→ returns a dataframe

but `df.iloc[:, 'Name']`
→ returns a series.

eg. df.

```
df = pd.DataFrame({
    'Name': ['A', 'B', 'C'],
    'Age': [20, 21, 22]},
    index = ['x', 'y', 'z'])
```

	Name	Age
x	A	20
y	B	21
z	C	22

column labels.

eg. df —

	name	age
x	A	10
y	B	20
z	C	30

row labels

uses labels

df.loc [rowlabel, column label]

=> >>> df.loc['x'] → 1st row.

>>> df.loc[:, 'Name'] → 1st column

both return as a series.

But, to get as Dataframe, →>>> df.loc[['x']] & df.loc[:, ['Name']]

Vimp

Now,

df.iloc[[0]] ⟺ df.loc[['x']]

uses indexes — index — for rows — label

simi, df.iloc[:, [1]] ⟺ df.loc[:, ['age']]

displays 'age' column